FIT3143 • Applied 2 • Tasks 1 and 2

GPU image rotation and responsible HPC for AI

CUDA design and performance • Evidence, trade-offs and accountable use

---

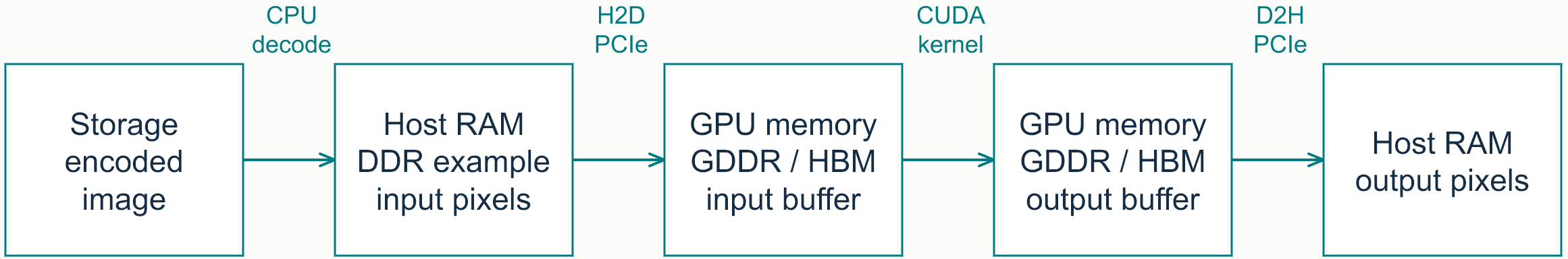
Savin Vindiv De Alwis · 35221631
sdea0018@student.monash.edu

Shamle Thilaksiri · 35512075
wthi0003@student.monash.edu

Host–GPU image data flow

CPU: load/decode, allocate, launch, save

GPU: read input, rotate, write output



H2D: host → device. D2H: device → host. Separate buffers; DMA (direct memory access) engines transfer bytes over PCIe.

GPU memory bandwidth differs from PCIe bandwidth. Pinned RAM supports DMA; pageable RAM may stage. Complete output before saving or reuse.

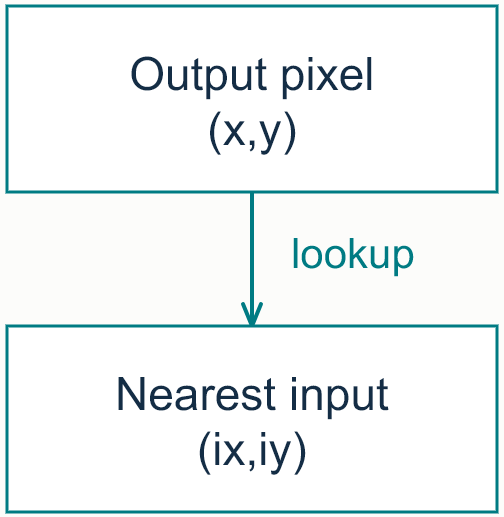
The CPU orchestrates I/O and launches; the GPU rotates decoded pixels.

Sources: [1] NVIDIA CUDA programming model; [2] NVIDIA CUDA Best Practices

Rotation as independent output work

Inverse mapping in y-down image coordinates

```
sx = c*(x-cx) - s*(y-cy) + cx;
sy = s*(x-cx) + c*(y-cy) + cy;
ix = floor(sx + 0.5);
iy = floor(sy + 0.5);
```



c = cos θ; s = sin θ; θ in radians; center = ((W−1)/2, (H−1)/2)

Forward [c s; −s c]; inverse [c −s; s c]. Positive θ is visually counterclockwise.

Fixed RGB canvas can crop; black borders. Gathering avoids scatter holes; nearest-neighbour can alias.

One thread owns one output pixel: gather a source value, then write once.

CUDA maps pixels to scheduled threads

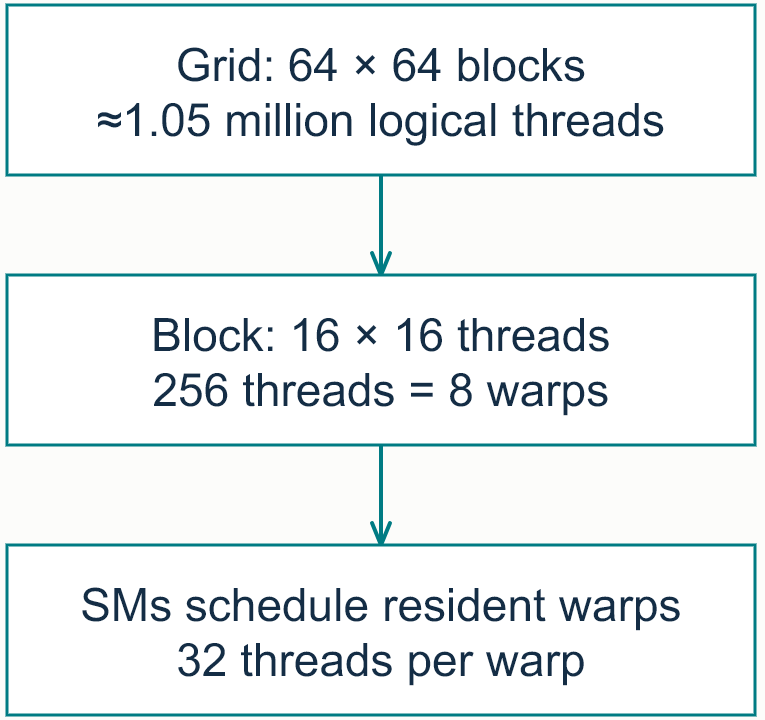
Host launch: asynchronous submission

```
dim3 block(16,16), grid(64,64);
rotate_rgb<<<grid,block>>>(
  d_in,d_out,1024,1024,c,s);
```

Device: global pixel index + edge guard. For other sizes, ceil-divide the grid. Check launch errors AND completion before reading/reusing output.

Not all threads run at once: finite SM resources execute queued blocks in waves.

SIMT = single instruction, multiple threads. A warp issues common instructions; divergent paths can reduce efficiency.



Appendix: full rotation, brightness, launch-shape and stream examples.

A logical grid exposes work; SMs execute resident warps with SIMT.

Sources: [1] NVIDIA CUDA programming model; [2] NVIDIA CUDA Best Practices

Throughput gains depend on the bottleneck

Why a GPU can be faster

Many execution units and high device-memory bandwidth process independent pixels concurrently.

Coalescing combines nearby addresses into fewer transactions. Rotated gathers can limit bandwidth; 32 × 8 keeps a warp on one row, unlike 16 × 16. Profile both.

Large images amortize setup; keeping pixels on the GPU avoids repeated copies. More resident warps hide latency, but registers and divergence constrain execution.

Compare completed equivalent work

Speedup = TCPU / TGPU,total. Same rotation/sampling on one CPU thread; also test a multithreaded CPU.

Streams can overlap independent images with separate pinned/device buffers and copy/compute support. Enqueue batches before waiting; extra buffers cost memory.

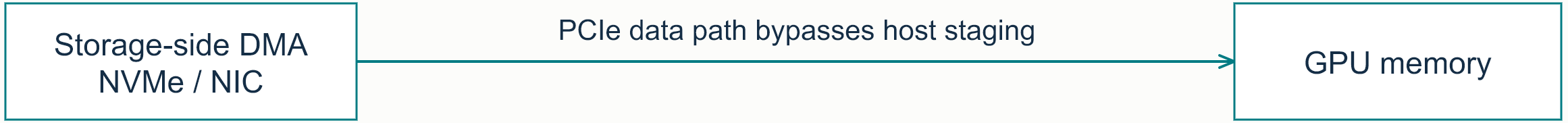
Time completed work: sequential setup + H2D + kernel + D2H, or the overlapped batch critical path. Amdahl bounds gains; no measured speedup is claimed.

GPUDirect Storage and rotation

Conventional storage path



GDS direct path (supported configuration)



CPU issues cuFile I/O. Fallback may stage through RAM; GDS does not decode images or accelerate kernel arithmetic.

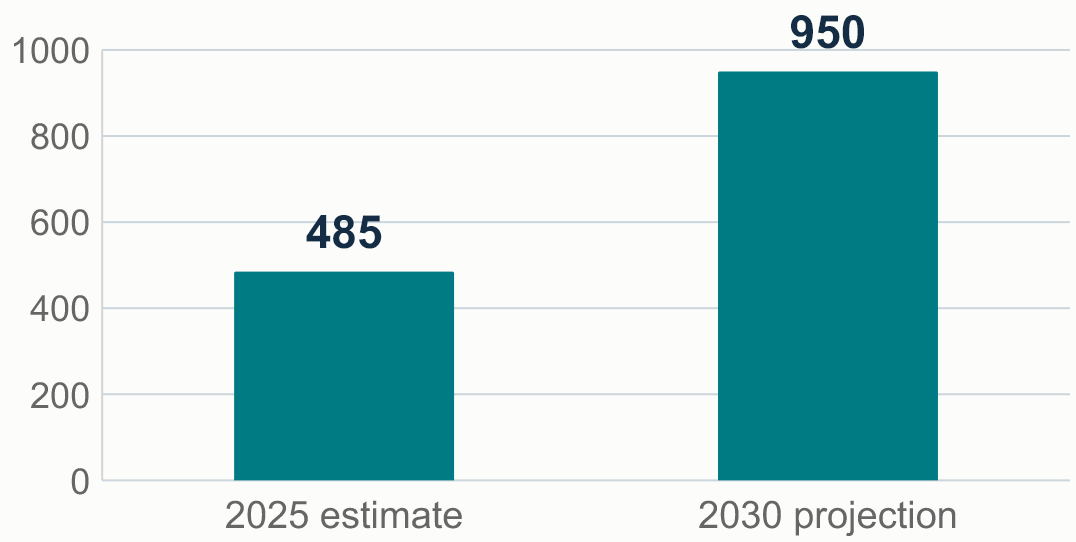
Use conventional copies for CPU-decoded / already-in-RAM pixels. Consider GDS for storage-bound GPU-ready batches only with supported GPU/filesystem/driver/topology and lower completed batch time.

GDS can reduce storage staging; it does not accelerate rotation arithmetic.

Sources: [3] NVIDIA GDS overview; [4] NVIDIA GDS design

Electricity demand and environmental costs

Global data-centre electricity (TWh/year)



IEA [6]: whole-sector context, not one AI job. Estimate vs projection; ≈96% growth. Full growth charts are in the appendix.

From cluster scaling to job-level impact

Data-parallel GPUs train different batches; gradient all-reduce synchronizes updates. Communication and stragglers limit scaling.

Energy = ∫ power dt; carbon ≈ Σ energy × local carbon intensity. Faster completion can still consume more energy.

Count CPUs, network/storage, cooling, tuning and failed runs. Water consumption varies by site/time; manufacture adds embodied impacts.

Green AI [10]: efficiency per useful output. Li et al. [9]: local water pressure. Track both; lower carbon alone can miss water costs.

Sources: [6] IEA 2026: sector estimates and projection; [9] Li et al. 2025 revision: water impacts; [10] Schwartz et al. 2019: efficiency reporting

Privacy, security and model fairness

Security: protect distributed intermediates

Workers exchange gradients and save checkpoints

↓

Least privilege, isolation, protected transport/storage

Zhu et al. [11]: input reconstruction from exposed gradients in studied settings. Protect intermediates; not every collective leaks.

Fairness: evaluate affected groups

Example: cluster-trained multilingual LLM coverage/harms differ by language

↓

Compare harmful-output rates by language + uncertainty, data-coverage fixes, reevaluation

Gallegos et al. [12]: metrics/datasets/mitigations differ. Reserve evaluation compute; GPU count cannot fix unrepresentative data.

Governance: provenance, permission, replica retention/deletion and dual-use review; named data/security owners.

Protect distributed data and evaluate harms; more compute is not a fairness guarantee.

Sources: [11] Zhu et al. 2019: exposed gradients; [12] Gallegos et al. 2024: fairness evaluation; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Compute concentration limits participation

Evidence and consequences

Ahmed & Wahed [13]: institutional participation. Strubell et al. [8]: workload cost. Different scopes reveal who can train, replicate and shape the AI research agenda.

Historical studies do not establish current global access.

NAIRR: >600 teams and >6,000 students [7]; US reach does not establish worldwide equality.

An equitable shared-cluster policy

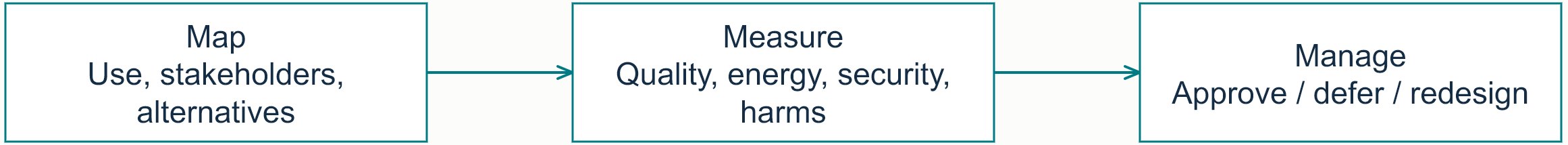
Publish GPU-hour rules; reserve capacity for smaller institutions and replication.

Offer training and appeals; audit waiting times and awards with privacy safeguards.

Trade-off: reservations can lower utilization or delay urgent jobs. Prevent starvation.

Sources: [7] NSF 2026: US programme reach; [8] Strubell et al. 2019: cost and equity; [13] Ahmed and Wahed 2020: compute divide

Approve scale with evidence and ownership



Govern throughout: accountable owner, rights, data rules and equitable access

Green AI [10] measures efficiency; Radovanovic et al. [14] turn forecasts into scheduling of flexible work. Measurement and action address different needs.

OECD [15]: rights and accountability. NIST [16]: operational risk decisions. Use an energy budget, named owner, safeguards and outcome review.

Defer only within deadlines, capacity, water/site and equitable waiting limits. Record urgent/forecast exceptions; prevent starvation.

NIST Govern spans iterative review; unacceptable risks require redesign or rejection.

Sources: [10] Schwartz et al. 2019: efficiency reporting; [14] Radovanovic et al. 2021: flexible scheduling; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Scale useful work, then verify the outcome

Task 1: conditional performance

Independent inverse-gather pixels suit CUDA; locality, transfers and SM scheduling determine speedup.

GDS helps only a compatible storage-bound path. Next: compile, compare CPU/GPU output and measure completed workloads.

Task 2: accountable scaling

Measure energy and useful output; protect data, evaluate group harms and allocate access transparently.

Next: test a real cluster job and review outcomes. Sector projections and proposed policies are evidence limits, not measured results.

Sources: [2] NVIDIA CUDA Best Practices; [3] NVIDIA GDS overview; [4] NVIDIA GDS design; [10] Schwartz et al. 2019: efficiency reporting; [16] NIST: Govern, Map, Measure, Manage

Detailed answers and supporting evidence

Task 1: implementation and analysis

Complete rotation and brightness examples, thread/grid calculations, host launch and stream completion.

Worked rotation, coordinate assumptions, launch-shape trade-offs, Amdahl bound, profiling and GPUDirect Storage decision.

Task 2: research and policy

Parallel training, energy/carbon calculations, environmental accounting and research limitations.

Risk owners, fairness example, allocation policy, framework roles, carbon-aware scheduling and cross-topic comparisons.

CUDA maps pixels to scheduled threads

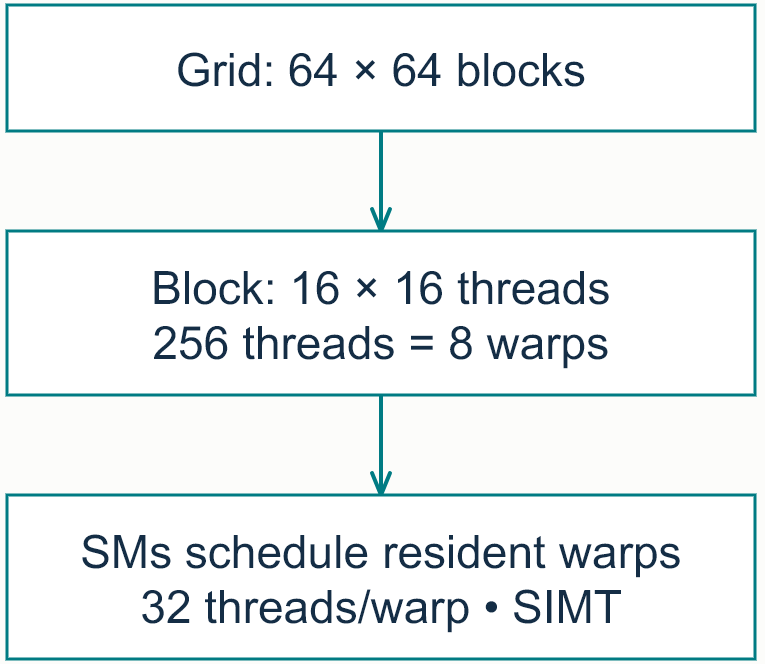
Host launch + device pixel indexing

```
dim3 block(16,16), grid(64,64);
rotate_rgb<<<grid,block>>>(
    d_in,d_out,1024,1024,c,s);
int x = blockIdx.x*blockDim.x + threadIdx.x;
int y = blockIdx.y*blockDim.y + threadIdx.y;
if (x >= W || y >= H) return;
```

Second kernel feature: clamp a brightness channel

```
int v = int(in[p]) + delta;
out[p] = v<0 ? 0 : (v>255 ? 255 : v);
```

After bounds checks: p = 3*(size_t(y)*W+x)+ch; ch ∈ [0,2]; delta ∈ [−255,255]. Separate RGB buffers. Uncompiled excerpts.

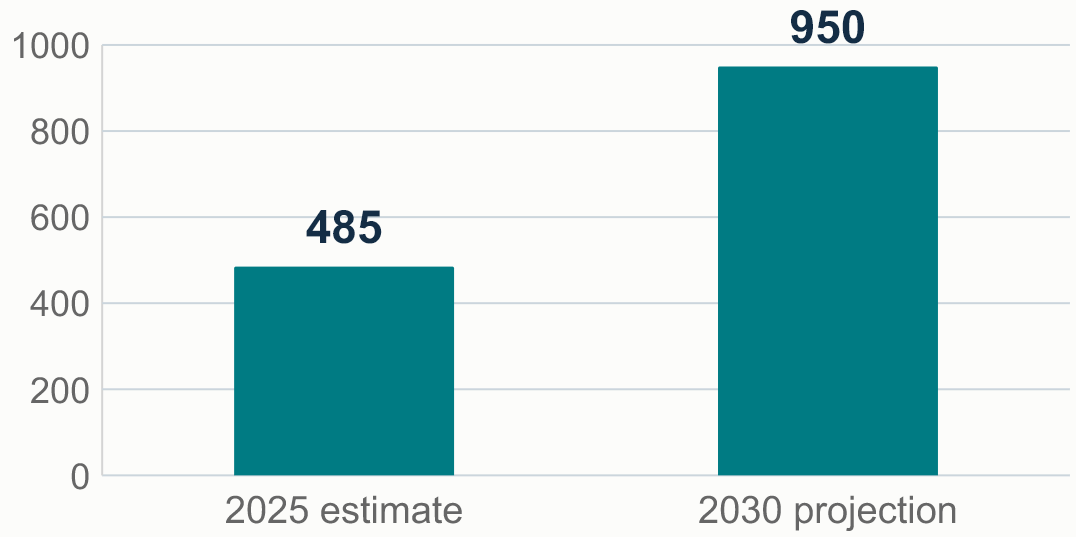


Residency hides latency; registers and divergence limit execution.

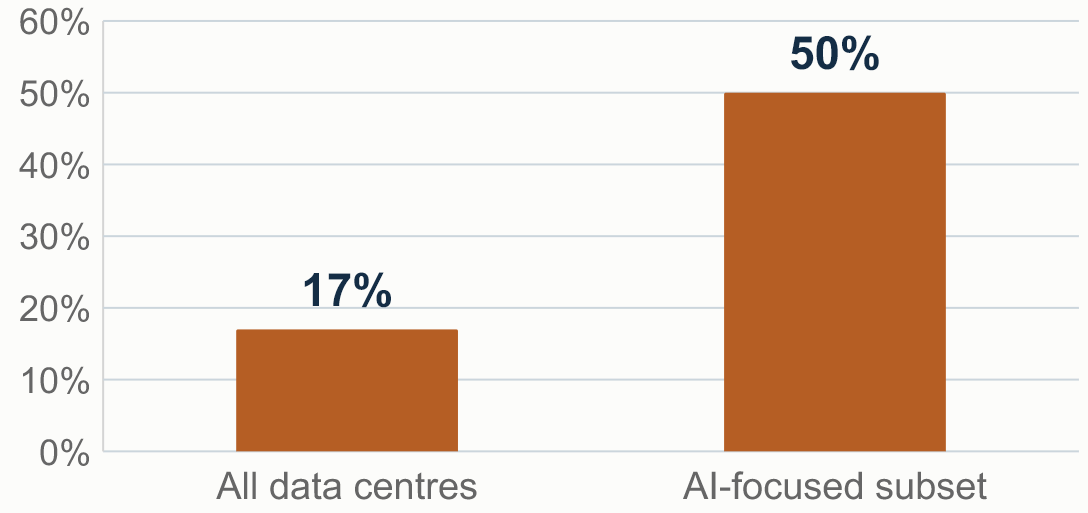
A logical grid exposes work; SMs execute resident warps with SIMT.

Electricity demand and environmental costs

Global data-centre electricity (TWh/year)



Electricity growth during 2025



IEA 2026: ≈96% increase to 2030. These cover the whole sector, not AI training alone; growth categories overlap and cannot be added.

Research comparison: Green AI [10] emphasizes efficiency; water research [9] adds site/time constraints. Track energy, carbon and local cooling impacts.

Sources: [6] IEA 2026: sector estimates and projection; [9] Li et al. 2025 revision: water impacts; [10] Schwartz et al. 2019: efficiency reporting

A 90-degree rotation example

Input

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19
20	21	22	23	24

$\theta = 90°$
$c = 0, s = 1$
center = (2,2)

Output: 90° counterclockwise

4	9	14	19	24
3	8	13	18	23
2	7	12	17	22
1	6	11	16	21
0	5	10	15	20

Output (0,1) → source (3,0) → value 3

The matrix values are illustrative pixel identifiers, not channel intensities.

CUDA launch and execution hierarchy

```
dim3 block(16,16);
dim3 grid(64,64);
rotate_rgb<<<grid,block>>>(
  d_in,d_out,1024,1024,c,s);
```

```
x = blockIdx.x*blockDim.x + threadIdx.x;
y = blockIdx.y*blockDim.y + threadIdx.y;
if (x >= W || y >= H) return;
```

1024 × 1024 output

64 × 64 = 4,096 blocks

16 × 16 = 256 threads/block

1,048,576 logical threads

32 threads/warp → 8 warps/block

One highlighted warp within a 16-wide block



Non-multiple dimensions: ceil-divide the grid and keep the bounds guard.

A grid specifies logical work; finite SMs schedule its blocks and warps.

CUDA rotation kernel

```
__global__ void rotate_rgb(const unsigned char* in,
    unsigned char* out, int W, int H, float c, float s) {
  int x = blockIdx.x*blockDim.x + threadIdx.x;
  int y = blockIdx.y*blockDim.y + threadIdx.y;
  if (x >= W || y >= H) return;
  float cx = (W-1)*0.5f, cy = (H-1)*0.5f;
  int ix = int(floorf(c*(x-cx)-s*(y-cy)+cx+0.5f));
  int iy = int(floorf(s*(x-cx)+c*(y-cy)+cy+0.5f));
  size_t dst = 3*(size_t(y)*W + x);
  if (ix >= 0 && ix < W && iy >= 0 && iy < H) {
    size_t src = 3*(size_t(iy)*W + ix);
    for (int ch=0; ch<3; ++ch) out[dst+ch] = in[src+ch];
  } else {
    for (int ch=0; ch<3; ++ch) out[dst+ch] = 0;
  }
}
```

One thread owns one output pixel. Guard partial edge blocks.

Inverse gather copies RGB; out-of-bounds indices write black.

Disjoint outputs need no atomics/barrier. Work and storage: O(W × H).

Separate RGB buffers; W/H > 0; valid sizes, c/s. Uncompiled and unbenchmarked.

Host orchestration and completion

```
dim3 block(16, 16);
dim3 grid(1u + (unsigned(W)-1u)/block.x,
          1u + (unsigned(H)-1u)/block.y);
cudaMemcpy(d_in, h_in, bytes,
           cudaMemcpyHostToDevice);
rotate_rgb<<<grid, block>>>(
    d_in, d_out, W, H, c, s);
cudaGetLastError();
cudaMemcpy(h_out, d_out, bytes,
           cudaMemcpyDeviceToHost);
```

Illustrative excerpt, not a complete program: W/H > 0; bytes = 3 × W × H checked in size_t; valid separate buffers; grid/device limits validated.

CPU loads/decodes, allocates host/device buffers, checks every API/launch/completion result, saves output and frees resources safely.

For this ordered synchronous-copy example, D2H supplies completed host data. A launch is asynchronous; launch-error checks alone do not establish successful completion.

Pinned host storage is page-locked and supports efficient DMA. Pageable memory may need runtime staging. Avoid excessive pinning.

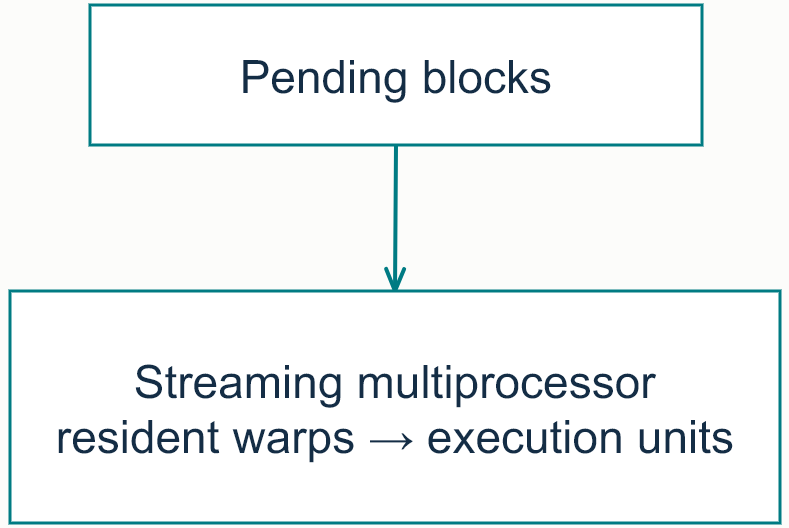
CUDA features that affect throughput

Example: independent brightness channels

```
size_t p = 3*(size_t(y)*W + x);
for (int ch=0; ch<3; ++ch) {
  int v = int(in[p+ch]) + delta;
  out[p+ch] = v<0 ? 0 : (v>255 ? 255 : v);
}
```

After coordinate/bounds checks; valid RGB buffers; delta ∈ [−255,255].

Resident warps hide latency; registers limit residency. Coalescing groups nearby accesses into fewer transactions; rotation can disrupt read locality.



SIMT = single instruction, multiple threads. A warp issues common instructions; divergent paths can reduce efficiency.

Concurrency helps only when memory access and resource use support it.

Sources: [1] NVIDIA CUDA programming model; [2] NVIDIA CUDA Best Practices

Launch shape and memory access

Choice	Mechanism	Performance implication
16 × 16 block	256 threads; each warp spans two 16-wide rows	Reasonable 2D starting point; row edges/alignment matter.
32 × 8 block	256 threads; each warp spans one 32-wide row	May improve row-oriented access; no universal winner.
Output gathering	Adjacent x threads write adjacent RGB pixels	Useful write locality; per-channel byte operations still require efficient transactions.
Rotated input reads	Angle changes the source addresses visited by a warp	Irregular gathers can reduce effective read bandwidth/cache reuse.
Higher residency	Registers, shared memory and block/thread limits constrain active warps	Helps hide latency; maximum occupancy is not automatically fastest.

Recompute grid for each block shape. Profile actual transactions, latency and registers before selecting a shape.

Expected speedup and its limits

Baseline: single CPU thread, same rotation and sampling

Also compare a multithreaded CPU; identical output and timing boundaries.

Speedup = TCPU / TGPU,total

TGPU,total = allocation/setup + H2D + kernel + D2H

Many GPU execution units and high device bandwidth favor parallel throughput. Large or resident images amortize setup; gathered reads may limit bandwidth.

Amdahl: S = 1 / (s + (1−s)/a). s is the unchanged serial share; a accelerates the remainder. s = 0.20 gives a 5× ideal ceiling before added transfers. Theoretical, not measured.

Expect gains for large or resident workloads; measure the whole pipeline.

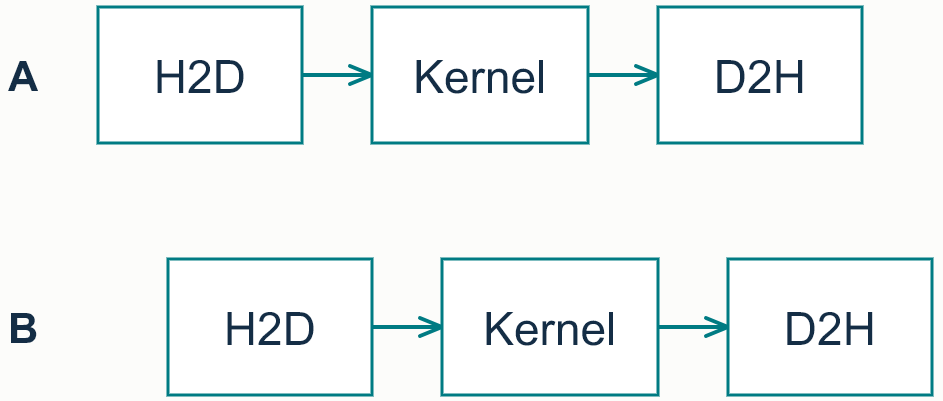
Streams, overlap and completed output

Ordered work in one stream

```
cudaMemcpyAsync(d_in, h_in, bytes,
  cudaMemcpyHostToDevice, stream);
rotate_rgb<<<grid, block, 0, stream>>>(
  d_in, d_out, W, H, c, s);
cudaMemcpyAsync(h_out, d_out, bytes,
  cudaMemcpyDeviceToHost, stream);
// After enqueueing all batch images:
cudaStreamSynchronize(stream);
```

Batch: enqueue all streams before waiting. Check every API, launch and completion result; submission is not completion.

Independent images / streams

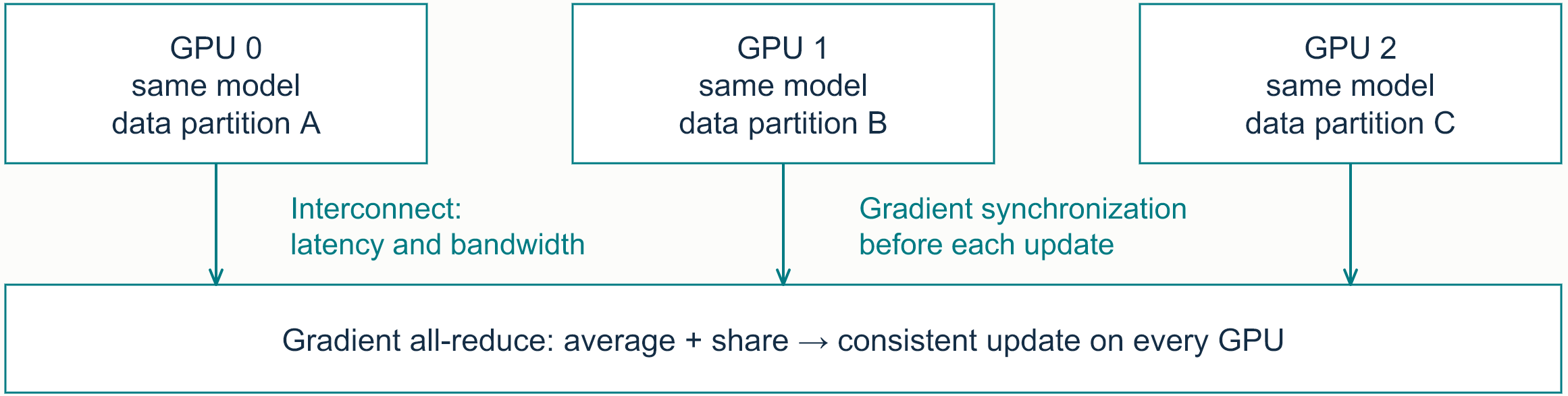


Possible overlap depends on supporting hardware and available resources.

Separate pinned/device buffers and non-default streams; synchronize before reuse. More buffers cost memory. Keep images on the GPU across operations to avoid repeated transfers.

Overlap can shorten the batch critical path; it changes scheduling, not total work.

Parallel training across a compute cluster



HPC combines processors, memory, networks and storage. Tensor/pipeline parallelism can also split a model.

More GPUs enable larger work; communication, serial work and imbalance limit speedup. Evaluate useful output against resource cost.

Scaling adds capacity and communication: ethical costs depend on useful work.

Sources: [5] NVIDIA Megatron parallelism; [2] NVIDIA CUDA Best Practices

A faster job can consume more energy

Assumed configuration	Completed runtime	Whole-job energy
Lower parallel scale; 1 kW	2.0 hours	1 × 2.0 = 2.0 kWh
Higher parallel scale; 2 kW	1.2 hours	2 × 1.2 = 2.4 kWh

$E = \int P(t)\,dt$ • Carbon ≈ Σ Einterval × carbon intensity

Illustrative constant power assumptions, not measurements. Same useful output is assumed; faster completion uses 20% more energy here.

Sources: [8] Strubell et al. 2019: cost and equity; [10] Schwartz et al. 2019: efficiency reporting; [2] NVIDIA CUDA Best Practices

Environmental accounting and evidence limits

Define what is counted

Energy (kWh) integrates power (kW) over hours. Operational emissions ≈ Σ energy in each interval × that interval's carbon intensity (kg CO_2e/kWh).

Report accelerators, host CPUs, storage/networking and allocated cooling overhead; include failed runs and repeated tuning when evaluating a project.

Hardware manufacture and disposal are embodied impacts beyond operational electricity. Water withdrawal and water consumption are different measures.

Compare the evidence

Strubell et al. quantify costs for selected historical NLP training/development workloads; those values are not universal 2026 model footprints.

Green AI motivates reporting efficiency alongside accuracy; Li et al. show why water varies with place/time. Carbon alone misses local water pressures.

IEA sector totals provide context, not causal attribution to one model. Forecasts depend on demand and infrastructure assumptions; local grid/cooling effects differ by site.

Sources: [6] IEA 2026: sector estimates and projection; [8] Strubell et al. 2019: cost and equity; [9] Li et al. 2025 revision: water impacts; [10] Schwartz et al. 2019: efficiency reporting

Governance controls and remaining risks

Risk / stakeholder	Proposed control	Residual limitation / owner
Dataset misuse / data subjects	Provenance, permission checks, retention limits and access logs	A licence alone may not justify every use; data steward reviews purpose.
Gradient/checkpoint exposure / participants	Least privilege, tenant isolation, protected transport and secure checkpoint storage	Authorized endpoints can still see sensitive values; security owner models insider risk.
Unsafe dual use / affected public	Authorized-use review, restricted access where justified and review of deployment risks	Controls cannot prevent every misuse; accountable owner weighs benefits/restrictions.
Distributed replicas / operators	Track copies, deletion obligations and incident response across workers	Replication increases the number of assets to protect; operator audits recovery.

These are proposed mitigations informed by research/frameworks, not a proof of privacy or a compliance certification.

Sources: [11] Zhu et al. 2019: exposed gradients; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Fairness evaluation with large-scale compute

Compute connection

An illustrative multilingual LLM service may have unequal performance or stereotyped outputs across language groups. Identify the actual harm before choosing a metric.

A cluster enables larger datasets and repeated training, but neither data volume nor GPU count guarantees representative data or equitable outcomes.

Reserve evaluation compute for affected groups; document dataset coverage and compare disaggregated error or harmful-output rates using a task-appropriate test.

Evidence and decision

Gallegos et al. separate metrics, evaluation datasets and mitigation stages. A single aggregate accuracy score can hide a relevant subgroup harm.

Specify justified acceptance criteria before training; examine uncertainty and small-group sample limits. Mitigate, reevaluate, and document unresolved harms.

Security and fairness have different objectives: encrypted gradients do not establish fair outputs. The source supports model-bias analysis, not a claim that HPC itself causes bias.

Sources: [12] Gallegos et al. 2024: fairness evaluation; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Unequal access to HPC resources

Evidence	What it shows	What it cannot establish
Ahmed & Wahed [13]	Historical research participation and compute concentration	Current worldwide access or a universal causal effect
Strubell et al. [8]	Selected NLP workload costs and financial barriers	The cost of every current AI workload

Current response: NAIRR supported >600 teams and >6,000 students

NSF, March 2026; US programme since 2024. Reach does not prove global equality.

Proposal: transparent GPU-hour allocation, smaller-institution support and appeals. Trade-off: reserved access can reduce immediate utilization or delay urgent jobs.

Shared capacity helps; allocation, expertise and reproducibility still need support.

Sources: [7] NSF 2026: US programme reach; [8] Strubell et al. 2019: cost and equity; [13] Ahmed and Wahed 2020: compute divide

Access research and allocation choices

Research comparison

Ahmed and Wahed study historical conference participation and a compute divide. Strubell et al. examine financial/resource barriers in selected NLP workloads.

The first concerns institutional participation; the second concerns workload cost. Together they motivate access support, but neither is a current worldwide access census.

NSF's March 2026 participation figures describe a US shared-resource response. Programme reach is not proof of equal allocation, global access or eliminated inequality.

Proposed shared-cluster policy

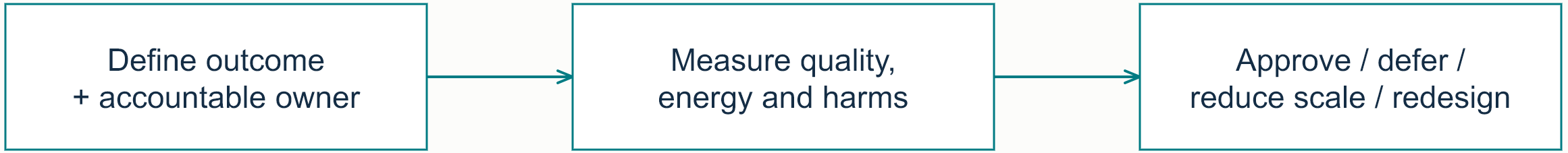
Publish eligibility, GPU-hour limits and allocation criteria; reserve some capacity for teaching, smaller institutions and independent replication.

Provide application support and training; allow appeals. Track applicant-to-award outcomes and waiting times across institution groups with privacy safeguards.

Trade-offs: reservations may reduce immediate utilization; queueing can delay urgent work; unrestricted access can conflict with security/budgets. Prevent repeated deferral or resource starvation.

Sources: [7] NSF 2026: US programme reach; [8] Strubell et al. 2019: cost and equity; [13] Ahmed and Wahed 2020: compute divide

An accountable HPC approval process



Green AI: efficiency evidence [10] • Carbon-aware scheduling: flexible work [14]

OECD: values and rights [15] • NIST: Govern, Map, Measure, Manage [16]

Use an energy budget, security/group evaluation and transparent access rules. Record owner, rationale, exceptions and outcomes.

Measurement informs action: shift flexible jobs to cleaner periods within deadlines/capacity. Forecast uncertainty, water stress and urgent access can change the decision.

Approve useful work with explicit budgets, safeguards and an accountable owner.

Sources: [10] Schwartz et al. 2019: efficiency reporting; [14] Radovanovic et al. 2021: flexible scheduling; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Framework roles and accountable decisions

Function / evidence	Concrete action	Decision / responsible role
Govern / OECD values + NIST	Assign ownership, data rules, access policy and escalation	Accountable owner accepts obligations and review process.
Map / NIST	Define use, stakeholders, misuse risks, deadline and alternatives	Data/risk stewards establish the application-specific scope.
Measure / Green AI + NIST	Track energy per useful outcome, model quality, group harms and scaling efficiency	Operator/evaluator provides evidence and uncertainty.
Manage / scheduling research + NIST	Approve, defer, reduce scale or redesign; enforce safeguards and review outcomes	Owner authorizes exceptions or rejects unresolved risks.

Our proposal combines measurement, scheduling and risk management. NIST Govern spans the other functions; risk management is iterative, not a certification.

Sources: [10] Schwartz et al. 2019: efficiency reporting; [14] Radovanovic et al. 2021: flexible scheduling; [15] OECD: rights, fairness and accountability; [16] NIST: Govern, Map, Measure, Manage

Carbon-aware scheduling under constraints



Research [14] demonstrates shifting temporally flexible workloads using forecasts while preserving daily capacity. It does not show every urgent GPU job can wait.

Proposed rule: defer only if deadline, energy budget, water/site limits and equitable waiting-time commitments remain satisfied; record exceptions.

No feasible window: reduce scale, redesign or approve a justified urgent exception. Prevent indefinite deferral. Data residency and transfer costs constrain moving work between sites.

Flexible jobs may move; urgent jobs, water constraints and access commitments can override deferral.

Sources: [9] Li et al. 2025 revision: water impacts; [14] Radovanovic et al. 2021: flexible scheduling; [16] NIST: Govern, Map, Measure, Manage

Conclusions and next measurements

GPU rotation

Inverse gathering exposes independent pixel work. SM scheduling, memory locality and transfer costs determine achieved performance.

Choose a CPU baseline and compare completed equivalent pipelines. GDS is conditional on storage bottlenecks, a compatible data path and verified support.

Next: compile the teaching examples, check output against a CPU reference and profile representative image sizes/angles/batches. No result is assumed.

Responsible scaling

More compute can enable valuable research while concentrating access and increasing resource demand. Approve scale for useful outcomes, with safeguards.

Combine job-level energy/resource evidence, separate security/fairness evaluation, transparent allocations and deadline-aware scheduling.

Next: measure a real cluster job’s energy, quality, scaling efficiency and allocation outcomes; revise policy when evidence or conditions change.

Sources: [2] NVIDIA CUDA Best Practices; [3] NVIDIA GDS overview; [4] NVIDIA GDS design; [10] Schwartz et al. 2019: efficiency reporting; [14] Radovanovic et al. 2021: flexible scheduling; [16] NIST: Govern, Map, Measure, Manage

Optional supporting material

Task 1: implementation expansions

The Task 1 answers already contain the rotation formula, worked calculation, kernel, host launch, launch-shape comparison and stream/overlap example.

The following slides expand coordinate conventions and complete source examples, then give an optional profiling plan and GDS background.

Task 2: supplementary comparisons

The Task 2 answers already contain energy/carbon calculations, research comparisons, governance and fairness decisions, access policy and accountable scheduling.

Additional parallelism and cross-topic comparison tables follow. Website references and one brief AI-use declaration complete the deck.

Rotation coordinates and sampling

Coordinate convention

Input/output: contiguous 8-bit RGB; positive width W and height H; x points right and y points down. Input and output buffers are distinct.

Rotate around cx = (W − 1)/2 and cy = (H − 1)/2. Positive θ is visually counterclockwise; the host supplies c = cos θ and s = sin θ in radians.

Forward image-coordinate rotation uses [c s; −s c]. Inverting it gives source offsets [c −s; s c] × output offsets, then restores the center.

Deliberate design choices

Nearest-neighbour indices: ix = floor(sx + 0.5), iy = floor(sy + 0.5). Copy three channels if the rounded indices are in bounds; otherwise write black.

The fixed W × H canvas can crop rotated corners. Nearest-neighbour sampling can alias; bilinear interpolation improves smoothness but adds source reads/arithmetic.

Forward scatter can leave holes or competing writes after rounding. Inverse gathering assigns every output exactly one writer. NPP offers library rotation with explicit interpolation/ROI choices.

Sources: [17] NVIDIA NPP: library alternative

Crafted CUDA example: RGB brightness

```
__global__ void brightness_rgb(const unsigned char* input,
                               unsigned char* output,
                               int width,
                               int height,
                               int brightness) {
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }
    const size_t offset =
        (size_t(y) * size_t(width) + size_t(x)) * 3;
    for (int channel = 0; channel < 3; ++channel) {
        const int value = int(input[offset + channel]) + brightness;
        output[offset + channel] =
            value < 0 ? 0 : (value > 255 ? 255 : value);
    }
}
```

Same operation on independent pixels: a data-parallel kernel.

Assume brightness in [−255,255]. Integer addition avoids uint8 wrap; clamp each channel to [0,255].

Adjacent threads touch adjacent pixel locations, providing useful locality.

Illustrative CUDA C++; uncompiled and unbenchmarked.

Crafted CUDA example: inverse mapping

```
__global__ void rotate_rgb(const unsigned char* input,
                           unsigned char* output,
                           int width,
                           int height,
                           float c,
                           float s) {
    const int x = blockIdx.x * blockDim.x + threadIdx.x;
    const int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= width || y >= height) {
        return;
    }

    const float cx = (width - 1) * 0.5f;
    const float cy = (height - 1) * 0.5f;
    const float sx = c * (x - cx) - s * (y - cy) + cx;
    const float sy = s * (x - cx) + c * (y - cy) + cy;
    const int ix = int(floorf(sx + 0.5f));
    const int iy = int(floorf(sy + 0.5f));
```

Global coordinates come from block/thread indices.

The guard handles partial edge blocks before any memory access.

The host computes sine/cosine once per angle.

Nearest-neighbour mapping uses the documented y-down convention.

Crafted CUDA example: safe output writes

```
    const size_t destination =
        (size_t(y) * size_t(width) + size_t(x)) * 3;

    if (ix >= 0 && ix < width && iy >= 0 && iy < height) {
        const size_t source =
            (size_t(iy) * size_t(width) + size_t(ix)) * 3;
        for (int channel = 0; channel < 3; ++channel) {
            output[destination + channel] = input[source + channel];
        }
    } else {
        for (int channel = 0; channel < 3; ++channel) {
            output[destination + channel] = 0;
        }
    }
}
```

RGB address = 3 × (row × width + column).

Every valid output gets three channels, including black borders.

No atomics, mutex or block barrier is needed: writers have disjoint destinations.

Total work Θ(W × H); input/output storage Θ(W × H) each.

Asynchronous work within one stream

```
cudaError_t enqueue_rotation(const unsigned char* h_input,
                             unsigned char* h_output,
                             unsigned char* d_input,
                             unsigned char* d_output,
                             size_t bytes, int width, int height,
                             float c, float s,
                             cudaStream_t stream) {
    const dim3 block(32, 8);  // 256 threads.
    const dim3 grid = rotation_grid(width, height, block);
    cudaError_t status = cudaMemcpyAsync(d_input, h_input, bytes,
                                         cudaMemcpyHostToDevice, stream);
    if (status != cudaSuccess) return status;
    rotate_rgb<<<grid, block, 0, stream>>>(d_input, d_output,
                                           width, height, c, s);
    status = cudaGetLastError();
    if (status != cudaSuccess) return status;
    return cudaMemcpyAsync(h_output, d_output, bytes,
                           cudaMemcpyDeviceToHost, stream);
}
```

Full enqueue function; rotation_grid uses the ceil-division shown in the host orchestration answer.

Pinned host buffers and checked sizes are preconditions.

Success means submitted work; wait before reading, reusing or freeing buffers.

On error, previously enqueued work may still be pending.

Fair speedup comparison and profiling

Comparable baselines

Primary baseline: one CPU thread performs the same RGB inverse mapping, nearest-neighbour rule and fixed canvas. Also compare a multithreaded CPU for a stronger practical baseline.

Use identical inputs/angles and verify identical or justified numerically equivalent outputs. Include file decoding/I/O in both paths or exclude it from both.

For the stated decoded-buffer comparison, include GPU allocation, H2D, launch, completed kernel and D2H; CPU timing covers the corresponding output allocation and rotation.

Measure the bottleneck

Report cold-start and warmed repeated runs separately. Use CUDA events for kernel elapsed time after completion and a host timer through final completion for total latency.

Vary image size, angle, block shape and batch size; record hardware, transfer mode, register use and effective memory/link bandwidth.

The sequential sum does not apply unchanged to overlapping streams: measure the completed batch critical path. Lower kernel time alone cannot establish end-to-end speedup.

Sources: [2] NVIDIA CUDA Best Practices

GDS decision for the proposed pipeline

Conventional decoded-image path

CPU decoding requires host-visible compressed bytes and produces pixels in RAM. Those pixels are copied to the GPU; the rotation kernel then runs.

A single small image already in RAM gains little from bypassing a storage step that is no longer on its path.

GDS does not decode an image, bypass an application's CPU transformation, or make GPU memory access itself faster.

Conditional batch alternative

Large storage-bound batches of GPU-ready pixels, or a compatible GPU decoding pipeline, may avoid host staging via cuFile and storage-side DMA.

Verify the GPU, storage/filesystem, software and PCIe topology support the direct path. Compatibility fallback uses host staging and is not evidence of direct DMA.

Adopt only if completed batch time/CPU overhead improve on the real workload; include extra buffers, decoding and storage bandwidth. Otherwise retain conventional copies.

Sources: [3] NVIDIA GDS overview; [4] NVIDIA GDS design

Parallelism, dependencies and scaling limits

Model	Partition and communication	Scaling limitation
Data parallel	Replicate model; partition batches; reduce/average gradients before a consistent update	Collective latency/bandwidth and stragglers.
Tensor parallel	Partition computations within a layer; exchange partial results	Frequent inter-device communication.
Pipeline parallel	Assign layers to stages; pass activations/gradients	Stage imbalance and pipeline bubbles.
Fixed workload	More GPUs reduce only the parallelizable portion	Serial work and communication bound speedup.
Larger workload	Scale capacity/model/data with added resources	A different claim from halving a fixed job’s runtime.

All-reduce is a collective result-sharing concept, not a claim that every AI framework uses the exact classroom MPI code.

Sources: [5] NVIDIA Megatron parallelism; [2] NVIDIA CUDA Best Practices

Research comparisons across all four topics

Topic	Comparison	Interpretation / limit
(a) Environment	Cost/efficiency research [8,10] versus water-footprint research [9]	Useful-output efficiency and local resource pressure need different measures.
(b) Governance	Gradient reconstruction [11] versus fairness evaluation survey [12]	Data exposure and unequal outcomes are distinct failure mechanisms.
(c) Access	Institutional participation [13] versus workload cost/equity [8]	Concentration and financial barriers reinforce a concern; scopes differ.
(d) Frameworks	Efficiency-reporting argument [10] versus operational scheduling [14]	Measurement motivates action; scheduling depends on flexibility and forecasts.

Current institutional evidence [6,7] and governance frameworks [15,16] complement the research; they do not replace its methodological limits.

Sources: [8] Strubell et al. 2019: cost and equity; [9] Li et al. 2025 revision: water impacts; [10] Schwartz et al. 2019: efficiency reporting; [11] Zhu et al. 2019: exposed gradients; [12] Gallegos et al. 2024: fairness evaluation; [13] Ahmed and Wahed 2020: compute divide; [14] Radovanovic et al. 2021: flexible scheduling

References

[1] NVIDIA, “Programming model,” CUDA Programming Guide, online documentation, accessed Oct. 8, 2026.
https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html

[2] NVIDIA, CUDA C++ Best Practices Guide, online documentation, accessed Oct. 8, 2026.
https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html

[3] NVIDIA, GPUDirect Storage Overview Guide, online documentation, accessed Oct. 8, 2026.
https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html

[4] NVIDIA, GPUDirect Storage Design Guide, online documentation, accessed Oct. 8, 2026.
https://docs.nvidia.com/gpudirect-storage/design-guide/index.html

References

[5] NVIDIA, “Parallelism strategies guide,” Megatron Core, online documentation, accessed Oct. 8, 2026.

https://docs.nvidia.com/megatron-core/developer-guide/latest/user-guide/parallelism-guide.html

[6] International Energy Agency, Key Questions on Energy and AI, executive summary, Apr. 16, 2026, CC BY 4.0, accessed Oct. 8, 2026.

https://www.iea.org/reports/key-questions-on-energy-and-ai/executive-summary

[7] U.S. National Science Foundation, “NAIRR at 2 years: Advancing American artificial intelligence innovation and leadership,” Mar. 19, 2026, accessed Oct. 8, 2026.

https://www.nsf.gov/cise/updates/nairr-2-years-advancing-american-artificial-intelligence

[8] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in Proc. ACL, 2019, pp. 3645–3650, doi: 10.18653/v1/P19-1355.

https://aclanthology.org/P19-1355/

References

[9] P. Li, J. Yang, M. A. Islam, and S. Ren, “Making AI less ‘thirsty’: Uncovering and addressing the secret water footprint of AI models,” arXiv:2304.03271v5, Mar. 26, 2025; first submitted 2023.

https://arxiv.org/abs/2304.03271v5

[10] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green AI,” arXiv:1907.10597, 2019.

https://arxiv.org/abs/1907.10597

[11] L. Zhu, Z. Liu, and S. Han, “Deep leakage from gradients,” arXiv:1906.08935v2, 2019.

https://arxiv.org/abs/1906.08935v2

[12] I. O. Gallegos et al., “Bias and fairness in large language models: A survey,” Computational Linguistics, vol. 50, no. 3, pp. 1097–1179, 2024, doi: 10.1162/coli_a_00524.

https://aclanthology.org/2024.cl-3.8/

References

[13] N. Ahmed and M. Wahed, “The de-democratization of AI: Deep learning and the compute divide in artificial intelligence research,” arXiv:2010.15581v1, 2020.

https://arxiv.org/abs/2010.15581v1

[14] A. Radovanovic et al., “Carbon-aware computing for datacenters,” arXiv:2106.11750v1, 2021.

https://arxiv.org/abs/2106.11750v1

[15] OECD, “OECD AI Principles,” adopted 2019, updated 2024, accessed Oct. 8, 2026.

https://www.oecd.org/en/topics/ai-principles.html

[16] NIST, AI Risk Management Framework 1.0 Playbook, framework 2023, current online resource, accessed Oct. 8, 2026.

https://airc.nist.gov/airmf-resources/playbook/

[17] NVIDIA, “Rotate,” NPP Image Geometry Transforms, CUDA 13.0.3 archive documentation, accessed Oct. 8, 2026.

https://docs.nvidia.com/cuda/archive/13.0.3/npp/image_geometry_transforms.html#rotate

AI use declaration

AI model and tool

OpenAI GPT-6, accessed through Codex.

How AI was used

Research and source checks; drafting and revision of explanations and CUDA teaching examples; creation of diagrams and charts; presentation formatting and assembly.